

1 Karp–Lipton Theorem

1.1 Preliminaries

- $P/poly$ is the class of languages decidable by polynomial-size circuit families. In particular, a language $L \in P/poly$ if there is a family of circuits $\{C_n\}_{n \geq 1}$ of size $poly(n)$ such that C_n decides L on inputs of length n .
- PH denotes the polynomial hierarchy. Refer to last lecture note or [AB09, Definition 5.3]
- A language L is in Σ_2^p if there exists a polynomial-time TM M and a polynomial p such that

$$x \in L \iff \exists u \in \{0, 1\}^{p(|x|)} \forall v \in \{0, 1\}^{p(|x|)} M(x, u, v) = 1.$$

- Similarly, $L \in \Pi_2^p$ if

$$x \in L \iff \forall u \in \{0, 1\}^{p(|x|)} \exists v \in \{0, 1\}^{p(|x|)} M(x, u, v) = 1.$$

1.2 Karp–Lipton Theorem

Theorem 1 (Karp–Lipton [KL82]). *If $NP \subseteq P/poly$, then $PH = \Sigma_2^p$.*

The goal is to show that $\Pi_2^p \subseteq \Sigma_2^p$. It suffices to show that Σ_2^p contains the Π_2^p -complete problem Π_2SAT , whose instances have the form $\forall u \exists v \varphi(u, v) = 1$.

Proof sketch. Suppose $NP \subseteq P/poly$. Then SAT has a polynomial-size circuit family. By the self-reducibility of SAT , the decision circuit can be converted, with polynomial overhead, into a circuit C' that outputs a satisfying assignment whenever one exists.

Consider an instance $\forall u \exists v \varphi(u, v) = 1$ of Π_2SAT . Under our assumption, there is a polynomial-size circuit C' that, given φ and u , outputs a witness v whenever such a witness exists. Therefore,

$$\forall u \exists v \varphi(u, v) = 1 \iff \exists C' \forall u \varphi(u, C'(\varphi, u)) = 1.$$

The circuit C' has polynomial size, so its description can be existentially quantified, and the remaining predicate can be checked in polynomial time. The right-hand side is therefore a Σ_2^p statement.

Hence $\Pi_2^p \subseteq \Sigma_2^p$. Taking complements gives $\Sigma_2^p \subseteq \Pi_2^p$, so $\Sigma_2^p = \Pi_2^p$. Hence the polynomial hierarchy collapses to Σ_2^p . Since equality of the second existential and universal levels collapses all higher levels, $PH = \Sigma_2^p$. $\square$

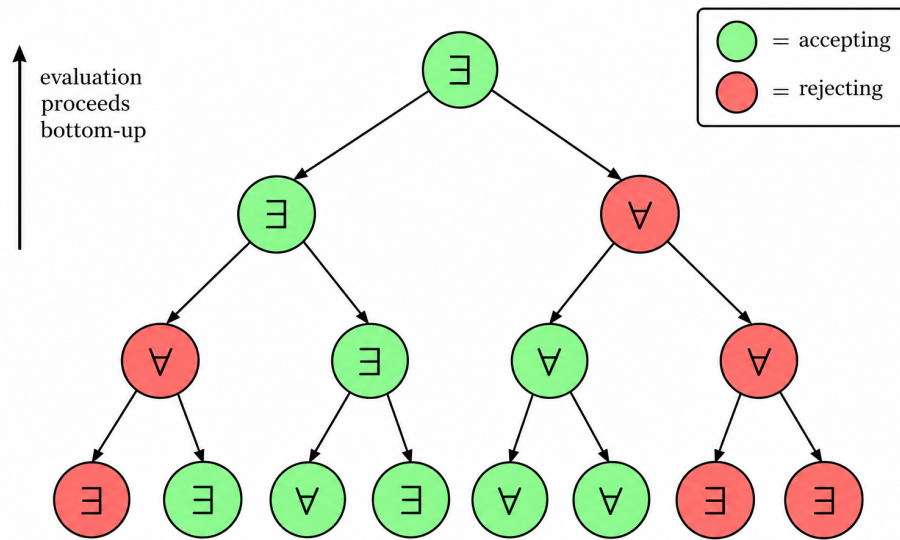


Figure 1: Bottom-up evaluation of an alternating Turing machine computation tree.

2 Alternating Turing Machine

Alternating Turing machines give a machine-based interpretation of these alternating quantifiers, generalizing nondeterministic Turing machines.

Definition 2 (Alternating Turing Machine). A t -tape alternating Turing machine has:

- State space $\mathbb{S} = \{(i, Q) : i \in [c], Q \in \{\exists, \forall\}\}$, where each state is labeled existential or universal.
- Tape alphabet $\mathcal{A} = \{0, 1, \square\}$.
- Transition function $\delta : \mathbb{S} \times \mathcal{A}^t \rightarrow \mathbb{S}^2 \times \mathcal{A}^{2t} \times \{L, R, S\}^{2t}$. Thus each non-halting configuration has two possible successor configurations.

Remark. An ordinary nondeterministic Turing machine is therefore a special case of an ATM in which every state is existential.

Computation tree. The execution of an ATM can be represented as a tree, where the root is the initial configuration and each node has its possible successor configurations as children. Transition proceeds from the root toward the leaves, while acceptance is evaluated from the leaves back toward the root. After labeling halting leaves as accepting or rejecting, an existential node accepts if at least one child accepts, while a universal node accepts only if all of its children accept.

For this lecture, the main reason to introduce alternating Turing machines is that the proof of Fortnow's Theorem passes through the class $\Sigma_2\text{TIME}$.

3 Fortnow's Theorem

Definition 3 ($\text{TISP}(T(n), S(n))$). $\text{TISP}(T(n), S(n))$ is the class of languages L for which there exists a deterministic Turing machine M that decides L using simultaneously $O(T(n))$ time and $O(S(n))$ space on inputs of length n .

Motivation.

- Is $\text{SAT} \in \text{L}$, where $\text{L} = \text{SPACE}(O(\log n))$? – We don't know.
- Is $\text{SAT} \in \text{SPACE}(n^{0.1})$? – We still don't know.
- However, if we restrict both time and space simultaneously, we can prove an impossibility.

Theorem 4 (Fortnow [For00] [FLVMV05]).

$$\text{SAT} \notin \text{TISP}(n^{1.1}, n^{0.1})$$

The proof is composed of the following three claims. See details in [AB09, Section 5.4]

Claim 5.

$$\text{SAT} \in \text{TISP}(n^{1.1}, n^{0.1}) \implies \text{NTIME}(n) \subseteq \text{TISP}(n^{1.2}, n^{0.2}).$$

The proof uses the Cook–Levin reduction. A nondeterministic computation running in time n can be reduced to a SAT formula of size $O(n \log n)$, while the reduction itself uses $O(n \log n)$ time and $O(\log n)$ space. Composing this reduction with the assumed SAT algorithm gives the slightly weaker bounds $n^{1.2}$ time and $n^{0.2}$ space.

Claim 6.

$$\text{TISP}(n^{12}, n^2) \subseteq \Sigma_2 \text{TIME}(n^8).$$

The proof is similar to the proofs of Savitch's Theorem and the PSAPCE-completeness of TQBF. See the next lecture's note for the full proof.

Claim 7.

$$\text{NTIME}(n) \subseteq \text{DTIME}(n^{1.2}) \implies \Sigma_2 \text{TIME}(n^8) \subseteq \text{NTIME}(n^{9.6}).$$

The proof uses a clever trick of padding. We again defer the proof the next lecture.

Combining the claims. Suppose for contradiction that $\text{SAT} \in \text{TISP}(n^{1.1}, n^{0.1})$. By the first claim, $\text{NTIME}(n) \subseteq \text{TISP}(n^{1.2}, n^{0.2})$, and hence also $\text{NTIME}(n) \subseteq \text{DTIME}(n^{1.2})$.

Padding by a factor of 10 in the exponent gives $\text{NTIME}(n^{10}) \subseteq \text{TISP}(n^{12}, n^2)$. The three claims give

$$\text{NTIME}(n^{10}) \subseteq \text{TISP}(n^{12}, n^2) \subseteq \Sigma_2 \text{TIME}(n^8) \subseteq \text{NTIME}(n^{9.6}).$$

This contradicts the nondeterministic time hierarchy theorem. Therefore $\text{SAT} \notin \text{TISP}(n^{1.1}, n^{0.1})$.

Acknowledgement

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [FLVMV05] Lance Fortnow, Richard Lipton, Dieter Van Melkebeek, and Anastasios Viglas. Time-space lower bounds for satisfiability. *Journal of the ACM (JACM)*, 52(6):835–865, 2005.
- [For00] Lance Fortnow. Time-space tradeoffs for satisfiability. *Journal of Computer and System Sciences*, 60(2):337–353, 2000.
- [KL82] Richard M. Karp and Richard J. Lipton. Turing machines that take advice. *L’Enseignement Mathématique*, 28(2):191–209, 1982.